



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure (P) – Defines how success is measured for the agent.
Environment (E) – The external world in which the agent operates.
Actuators (A) – The mechanisms used by the agent to perform actions.
Sensors (S) – The mechanisms used by the agent to perceive the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width` → Grid width
`self.height` → Grid height
`self.walls` → Locations of obstacles
`self.food_positions` → Locations of food items
`self.agent_pos` → Current agent location
`self.opponents` → Positions of other agents
`self.score` → Current performance value
`self.steps` → Number of actions performed
`self.collision` → Collision status

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Answer: Multi-Agent Environment

Justification:

The environment contains:

```
self.opponents = []
```

and opponent movement logic:

```
for op in self.opponents:  
    move = random.choice(['Up', 'Down', 'Left', 'Right', 'Stay'])
```

The opponents act independently and can affect the agent by causing collisions:

```
if op == self.agent_pos:  
    self.score -= 50
```

Therefore, this is a multi-agent environment.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of observations received by an agent through its sensors over time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensor (S) component
because it provides environmental information to the agent.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

All possible future states

Therefore, the agent has incomplete knowledge of the environment.

Task 1.3: Action Execution (`execute_action`)**7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?**

Answer: Performance Measure (P)

The score is changed:

`self.score -= 5`

The score measures how well the agent performs.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

The performance measure should evaluate the result of actions in the environment rather than how much computation the agent performs.

For example:

Good measure:

Collecting food (+20)

Avoiding collisions

Increasing score

Bad measure:

Number of calculations performed

Processing time used

An intelligent agent should be rewarded for achieving goals, not for spending more computational resources.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.
9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

If toxic traps exist but are not included in:

`get_percept()`

the environment becomes more partially observable.

The agent cannot detect danger before moving onto a trap, so it must make decisions with incomplete information.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding:

```
'smells_toxin': tuple(self.agent_pos) in self.toxic_traps
```

improves the agent's rational decision-making because it provides extra information about dangerous states.

The agent can:

- Avoid toxic locations
- Choose safer paths
- Maximize expected utility
- Reduce unnecessary score penalties

Therefore, the additional sensor helps the agent select better actions.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World exploit occurs when the performance measure is poorly designed.

Example:

A vacuum agent may repeatedly clean the same already-clean square because the scoring system rewards cleaning actions rather than actual improvement of the environment.

This shows that a faulty metric can encourage the agent to exploit the scoring system instead of behaving intelligently.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING